DOKUMEN PERANCANGAN PROYEK · OPEN SOURCE

Lapisan Observabilitas dan Debugging untuk Lightpanda Browser

Agent Observability and Debugging Layer: tracer CDP eksternal, format JSONL, penampil statis, dan sidecar MCP.

Versi dokumen	v1.1
Tanggal	29 Oktober 2026
Status	Draf revisi
Pemirsa	Penggagas proyek, kontributor potensial, peninjau teknis
Versi Lightpanda acuan	1.0.2 (rilis 23 Oktober 2026). Bukti M0 pada Bab 5 memakai nightly 1.1.0 dan wajib diulang pada 1.0.2 sebelum M1 (lihat M0b, Bab 6).

Cara membaca status verifikasi pada dokumen ini. Label **Terverifikasi (M0)** berarti event atau perintah tersebut terbukti bekerja pada uji M0, Bab 5. Label **Belum terverifikasi** berarti klaimnya masuk akal tetapi belum dibuktikan, dan uji pembuktiannya dituliskan sebagai bagian dari M0b. Fitur berlabel **Belum terverifikasi** tidak boleh ditulis di README sebagai fitur yang sudah berjalan.

Dokumen ini adalah revisi terkendali dari draf v1.0. Riwayat perubahannya ada pada halaman 2. Isi Bab 1 sampai Bab 4 menyatakan desain yang diusulkan; Bab 5 menyatakan bukti pengujian yang sudah dilakukan; Bab 6 sampai Bab 10 menyatakan rencana, strategi, risiko, dan langkah selanjutnya.

Riwayat perubahan

Versi	Tanggal	Perubahan
v1.0	29 Oktober 2026	Draf awal.
v1.1	29 Oktober 2026	Perbaikan dari ulasan teknis dan bukti M0: (1) kalimat pada Bab 1.3 yang menyatakan rantai debugging Lightpanda lebih matang dari Chrome diperbaiki menjadi kurang matang; (2) karakter non-Indonesia pada tabel risiko dihapus; (3) nama tool MCP diseragamkan menjadi `get_trace_summary` dan `annotate_trace` di seluruh dokumen; (4) diagram arsitektur diperbaiki — tracer adalah klien CDP di luar proses Lightpanda, bukan hook di dalam kode Zig; (5) ditambahkan kewajiban handshake Target dan `sessionId`, berdasarkan kegagalan `BrowserContextNotLoaded` pada M0; (6) klaim waterfall diperbaiki — detail `timing` CDP bernilai -1 pada M0, sehingga durasi dibangun dari stempel waktu tracer; (7) status verifikasi setiap event dinyatakan jujur — `Runtime.exceptionThrown` dan `Log.entryAdded` belum terbukti dan tidak dijanjikan pada MVP; (8) ditambahkan Bab 3.6 tentang redaksi dan privasi; (9) ditambahkan diferensiasi terhadap fitur `agent --trace` bawaan Lightpanda; (10) definisi benchmark overhead diperketat; (11) janji `file://` untuk penampil dan ekspor HAR penuh diturunkan menjadi klaim yang dapat diuji. Tidak ada perubahan pada arah proyek: tracer CDP eksternal, format JSONL, penampil statis, dan sidecar MCP.

1. Ringkasan Proyek

1.1 Latar Belakang

Lightpanda adalah browser headless (tanpa antarmuka grafis) yang dibangun dari nol menggunakan bahasa pemrograman Zig. Browser ini dirancang khusus untuk otomasi mesin, pengambilan data web (web scraping), dan AI Agent, bukan untuk manusia. Karena tidak memiliki mesin rendering grafis, Lightpanda jauh lebih efisien secara sumber daya dibandingkan Chrome Headless.

Berdasarkan benchmark resmi Lightpanda, dengan 25 tugas paralel yang mengambil 933 halaman demo, Lightpanda menyelesaikan pekerjaan dalam sekitar 5 detik dengan puncak memori 123 MB. Sebagai perbandingan, Chrome membutuhkan sekitar 46 detik dan 2 GB memori, yaitu sekitar 9 kali lebih lambat dan 16 kali lebih boros memori. Angka-angka ini berasal dari benchmark Lightpanda sendiri dan berlaku untuk beban kerja benchmark tersebut, bukan untuk semua situs.

Lightpanda merilis versi 1.0 pada 2 Oktober 2026, lulus lebih dari 1,7 juta sub-tes Web Platform Tests, dan menerapkan CORS secara default. Proyek ini bersifat open source dengan lisensi AGPL-3.0. Rilis 1.0.2 tersedia sejak 23 Oktober 2026 dan menjadi versi acuan dokumen ini.

1.2 Posisi Proyek

Proyek ini **bukan** upaya menyalin ulang Lightpanda dari nol. Proyek ini membangun di atas kemampuan Lightpanda yang sudah ada, dengan memilih satu arah vertikal yang belum tercakup secara memadai, lalu mengembangkannya secara open source untuk komunitas.

Arah yang dipilih adalah:

> **Membangun “Lapisan Observabilitas dan Debugging yang Ramah Agent” (Agent Observability and Debugging Layer) untuk Lightpanda.**

1.3 Alasan Pemilihan Arah

Lightpanda sudah memiliki kemampuan eksekusi yang kuat (mode `fetch`, `serve`, `agent`, dan `mcp`), tetapi aspek observabilitas dalam skenario agent masih lemah:

- Ketika Agent mengendalikan browser melalui CDP atau MCP, pengembang sulit mengetahui apa yang terjadi di setiap langkah: waktu pemuatan halaman, durasi eksekusi JavaScript, daftar permintaan jaringan, dan urutan kegagalan.
- Kegagalan Agent sering kali bersifat senyap (timeout, selector tidak ditemukan, error JavaScript yang tertelan), sehingga proses debugging menjadi mahal.
- Chrome DevTools Protocol (CDP) menyediakan domain untuk tracing dan debugging, tetapi Lightpanda sebagai browser yang dibangun dari nol memiliki rantai alat debugging yang jauh **kurang** matang dibandingkan Chrome. Celah inilah yang diisi proyek ini.

Arah ini tidak bersaing langsung dengan tim inti Lightpanda yang sedang mengerjakan “browser yang lebih cepat dan lebih ringan”. Proyek ini melengkapi ekosistem dan secara alami cocok untuk kontribusi komunitas open source.

2. Tujuan dan Non-Tujuan

2.1 Tujuan

No	Tujuan	Kriteria terukur
T1	Menyediakan ekspor aliran event terstruktur untuk sesi CDP Lightpanda	Output format JSONL, mencakup 4 kategori: navigation, network, console, dan error. Kategori error hanya dihitung tercapai bila M0b membuktikan `Runtime.exceptionThrown` untuk exception halaman yang tidak tertangkap (lihat Bab 5).
T2	Menyediakan web UI ringan untuk pemutaran ulang (replay) pasca-kejadian	Aplikasi satu halaman, tanpa database backend, membaca file JSONL melalui pemilih file, dan dijalankan lewat server statis lokal (lihat Bab 4.1, F3).
T3	Terintegrasi dengan mode MCP agar AI Agent dapat “melihat catatan operasinya sendiri”	Tool MCP `get_trace_summary` dan `annotate_trace` tersedia pada sidecar MCP (lihat Bab 4.1, F4). Nama tool ini berlaku di seluruh dokumen.
T4	Overhead performa di bawah 5%	Benchmark sesuai definisi pada Bab 3.7: median wall-clock 5 ulangan dengan tracer aktif tidak lebih dari 5% di atas median tanpa tracer, pada beban kerja dan versi Lightpanda yang sama, dengan telemetri Lightpanda dimatikan.

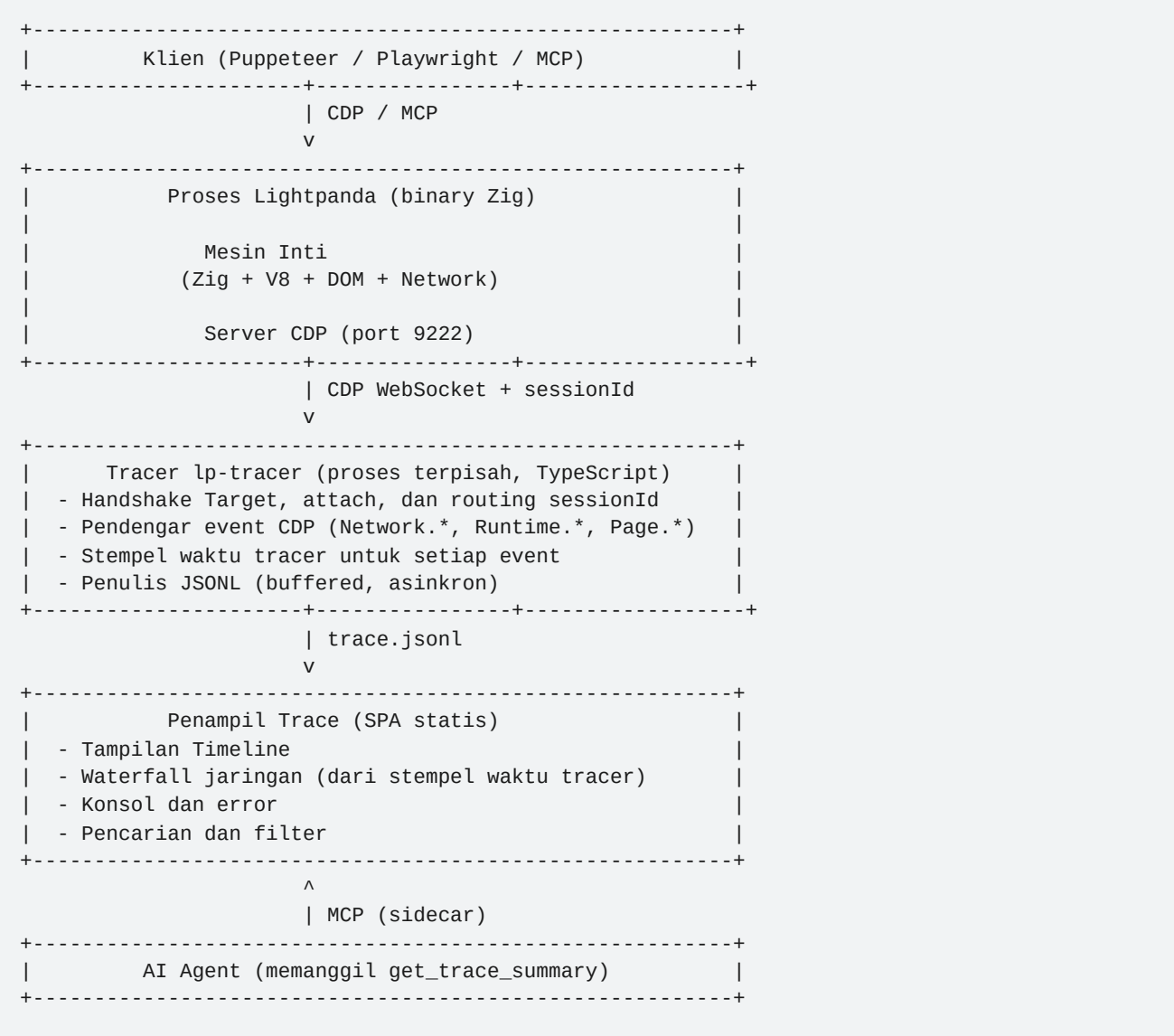
2.2 Non-Tujuan

- Tidak mengubah logika inti mesin rendering atau JavaScript Lightpanda.
- Tidak mengimplementasikan pengganti penuh frontend Chrome DevTools.
- Tidak menyediakan fitur kolaborasi real-time atau penyimpanan cloud (prioritas pada file lokal).
- Tidak menyentuh debugging terkait rendering visual (Lightpanda sendiri tidak melakukan rendering).
- Tidak merekam isi body permintaan atau respons secara default, dan tidak merekam rahasia apa pun (lihat Bab 3.6).
- Tidak menyediakan diff DOM yang detail pada MVP. Event `DOM.documentUpdated` hanya dicatat sebagai penanda kasar bila diterima.

3. Solusi Teknis

3.1 Arsitektur Keseluruhan

Tracer berjalan sebagai **proses terpisah** di luar proses Lightpanda. Ia tidak disisipkan ke dalam kode Zig Lightpanda dan tidak menautkan kode Lightpanda.



3.2 Keputusan Teknis Kunci

Keputusan 1: Injeksi melalui lapisan CDP, bukan modifikasi mesin. Lightpanda sudah mengekspos server CDP. Protokol CDP memuat domain `Network.enable`, `Runtime.enable`, dan `Page.enable` yang dapat digunakan untuk berlangganan permintaan jaringan, pesan konsol, dan event navigasi. Lapisan tracing berperan sebagai klien CDP yang terhubung ke port CDP Lightpanda, mendengarkan dan mencatat event tersebut, tanpa mengubah kode sumber Zig Lightpanda. Keuntungannya: tanpa intrusi (tidak perlu memelihara fork Zig); portabel secara prinsip (tracer yang sama secara teoritis dapat digunakan untuk Chrome Headless, meskipun perilaku dan kelengkapan event-nya berbeda dan harus diuji terpisah); dan ambang kontribusi yang rendah (pengembang komunitas hanya perlu menguasai TypeScript atau JavaScript).

Keputusan 2: Format JSONL untuk trace. Satu objek JSON per baris, berisi `ts` (stempel waktu), `seq` (nomor urut), `session` (pengenal sesi), `type` (jenis event), dan `data` (payload event). JSONL secara alami mendukung penulisan streaming dan parsing baris per baris, cocok untuk sesi agent yang berjalan lama. Skema lengkap ada pada Lampiran A.

Keputusan 3: Integrasi MCP secara sidecar. Lightpanda memiliki perintah `mcp` bawaan untuk menjalankan server MCP. Tracer tidak menggantikan server MCP tersebut, melainkan berjalan sebagai proses sidecar independen yang mengekspos dua tool MCP tambahan untuk Agent. Agent dapat memanggil `get_trace_summary` setelah menyelesaikan tugas untuk meninjau kembali apa yang telah dilakukannya, dan `annotate_trace` untuk menandai langkah penting. Sidecar membaca file JSONL yang sama dengan yang ditulis tracer, sehingga tidak ada format kedua.

Keputusan 4: Waterfall dibangun dari stempel waktu tracer, bukan dari field `timing` CDP. Pada M0, field `timing` di dalam `Network.responseReceived` bernilai -1 untuk seluruh sub-field (DNS, connect, SSL, send, receive), dan `encodedDataLength` bernilai 0. Karena itu tracer mencatat waktu diterimanya setiap event menurut jam tracer sendiri, dan durasi permintaan dihitung sebagai selisih antara event `Network.requestWillBeSent`, `Network.responseReceived`, dan `Network.loadingFinished` untuk `requestId` yang sama. Waterfall pada penampil menampilkan durasi total dan fase kasar (menunggu respons, selesai) saja. Breakdown DNS, TLS, dan ukuran byte tidak ditampilkan sebagai data CDP; bila suatu saat Lightpanda mengisi field tersebut, penampil boleh menampilkannya dengan label sumber yang jelas.

Keputusan 5: Implementasi referensi memakai WebSocket native, dengan handshake Target yang eksplisit. Implementasi referensi memakai WebSocket native Node.js (tersedia sejak Node 22) dan melakukan handshake Target secara eksplisit (lihat Bab 3.3). Pustaka `chrome-remote-interface` hanya boleh dipakai bila terbukti mendukung routing `sessionId` dengan benar pada Lightpanda; hal ini harus dibuktikan pada M1 sebelum pustaka tersebut diadopsi, karena kegagalan routing `sessionId` membuat tracer tidak menerima event apa pun.

3.3 Handshake Target (Wajib)

Koneksi WebSocket mentah ke endpoint browser (`ws://127.0.0.1:9222/`) **tidak cukup**. Pada M0, perintah `Network.enable`, `Runtime.enable`, dan `Page.navigate` yang dikirim tanpa sesi target dijawab dengan error `BrowserContextNotLoaded` (kode -31998) dan tidak ada event yang diterima. Tracer wajib melakukan urutan berikut sebelum mengirim perintah `enable`:

1. `Target.setDiscoverTargets` dengan `{ discover: true }`.
2. `Target.getTargets`, lalu pilih target bertipe `page` bila ada.
3. Bila daftar target kosong, `Target.createTarget` dengan `{ url: "about:blank" }`.
4. `Target.attachToTarget` dengan `{ targetId, flatten: true }`, dan simpan `sessionId` yang dikembalikan.
5. Kirim `Page.enable`, `Network.enable`, dan `Runtime.enable` dengan menyertakan `sessionId` pada setiap pesan. Event yang diterima juga membawa `sessionId` yang sama.
6. Kirim `Page.navigate` dengan `sessionId` yang sama.

Kegagalan pada langkah 4 adalah kegagalan fatal untuk sesi tersebut: tracer harus berhenti dengan pesan error yang jelas, bukan berjalan tanpa event. Perilaku ini harus tercakup oleh pengujian M0b dan M1. Domain `Log` tidak termasuk dalam urutan ini (lihat Bab 3.5).

Tracer yang berjalan berdampingan dengan Puppeteer atau Playwright harus diuji secara eksplisit (lihat M0b): setiap klien memakai target dan sesi masing-masing, dan tracer tidak boleh menutup atau mengambil alih target milik klien lain. Pada nightly yang diuji M0, server menerima koneksi CDP kedua tanpa mematikan koneksi pertama, tetapi attach target kedua secara penuh belum diuji.

3.4 Tumpukan Teknologi

Komponen	Pilihan teknologi	Alasan
Inti tracer	TypeScript (Node.js), WebSocket native	Tanpa dependensi CDP tambahan pada implementasi referensi; routing `sessionId` terlihat eksplisit pada kode
Format trace	JSONL	Streaming, dapat ditambahkan (append), mudah di-parse
Penampil	SvelteKit (ekspor statis)	Ringan, tanpa backend, dijalankan lewat server statis lokal
Sidecar MCP	TypeScript (MCP SDK)	Menggunakan ulang aliran event dan file JSONL inti tracer
Pengemasan	Docker dan paket npm mandiri	Mencakup skenario container dan pengembangan lokal

Versi yang dikunci untuk pengujian integrasi M1 adalah Lightpanda 1.0.2 dan Node.js 22 atau lebih baru. Perubahan versi dicatat pada Bab 6.

3.5 Batas Cakupan Event

Tabel berikut menyatakan status setiap event pada akhir M0. Status **Belum terverifikasi** berarti event tersebut direncanakan, tetapi tracer dan penampil harus menanganinya sebagai event yang mungkin tidak pernah datang, dan README tidak boleh mengklaimnya sebagai fitur yang sudah berjalan.

Kategori event	Sumber CDP	Status
Navigasi	`Page.frameNavigated`	Terverifikasi (M0)
Permintaan jaringan	`Network.requestWillBeSent`	Terverifikasi (M0)
Respons jaringan	`Network.responseReceived`	Terverifikasi (M0)
Selesai memuat	`Network.loadingFinished`	Terverifikasi (M0)
Kegagalan jaringan	`Network.loadingFailed`	Belum terverifikasi (tidak ada kegagalan pada beban uji M0; uji M0b memakai URL yang sengaja gagal)
Konsol	`Runtime.consoleAPICalled`	Terverifikasi (M0); pemetaan level `warn` dan `error` belum dikonfirmasi
Exception JavaScript	`Runtime.exceptionThrown`	Belum terverifikasi (uji M0 dengan throw lewat `Runtime.evaluate` tidak menghasilkan event; uji M0b memakai exception pada halaman yang sedang dimuat)
Entri log	`Log.entryAdded`	Belum terverifikasi (`Log.enable` diterima, tetapi tidak ada entri pada M0; domain Log tidak dipakai pada MVP)
Perubahan dokumen	`DOM.documentUpdated`	Teramati sekali pada M0; dicatat sebagai penanda kasar saja

3.6 Redaksi dan Privasi (Wajib)

File trace adalah catatan yang mudah bocor. Aturan berikut berlaku secara default dan tidak dapat dimatikan secara diam-diam:

1. **Header sensitif selalu disensor.** Nilai header `authorization`, `cookie`, `set-cookie`, `proxy-authorization`, dan `x-api-key` (tanpa membedakan huruf besar dan kecil) diganti dengan `[REDACTED]` sebelum ditulis. Nama header tetap dicatat agar pola permintaan terlihat.
2. **Body tidak direkam secara default.** Isi body permintaan dan respons, termasuk yang diperoleh lewat `Network.getResponseBody`, tidak ditulis ke JSONL. Pengambilan body hanya tersedia lewat flag eksplisit `--include-bodies` yang disertai peringatan pada CLI, dan tidak pernah menjadi default pada contoh maupun pengujian.
3. **Argumen konsol dipotong.** Setiap argumen `Runtime.consoleAPICalled` dipotong sampai 500 karakter, dan satu event dibatasi 2.000 karakter. URL dicatat utuh kecuali mengandung parameter yang tampak seperti token (`token`, `key`, `secret`, `password`, `auth`), yang nilainya disensor.
4. **Ringkasan MCP memakai trace yang sudah disensor.** Tool `get_trace_summary` hanya membaca file JSONL yang sudah melewati redaksi. Ia tidak pernah membaca ulang data mentah dari browser.
5. **Anotasi adalah tanggung jawab pemanggil.** Teks yang ditulis lewat `annotate_trace` disimpan apa adanya, karena teks tersebut ditulis secara sadar oleh agent atau pengguna. Dokumentasi sidecar menyatakan bahwa anotasi tidak boleh berisi rahasia.
6. **Batas ukuran dan rotasi.** Satu file trace dibatasi 50 MB secara default (dapat diubah lewat flag). Saat batas tercapai, tracer berhenti menulis dengan pesan yang jelas, bukan menimpa data lama secara diam-diam. Izin file trace dibuat 0600 pada sistem yang mendukungnya.

3.7 Definisi Benchmark Overhead (T4)

Klaim overhead di bawah 5% hanya bermakna bila diukur sebagai berikut:

- **Beban kerja:** sekumpulan URL tetap yang dijalankan secara lokal atau terhadap server demo yang sama, dengan urutan navigasi yang sama, pada setiap ulangan.
- **Pembanding:** median wall-clock 5 ulangan dengan tracer terpasang dan aktif, dibandingkan dengan median 5 ulangan tanpa tracer, pada versi Lightpanda yang sama dan perintah yang sama.
- **Kondisi:** telemetri Lightpanda dimatikan (`LIGHTPANDA_DISABLE_TELEMETRY=true`) pada kedua sisi, agar telemetri tidak mengacaukan perbandingan. CPU tracer dihitung terpisah dari proses Lightpanda dan dilaporkan bersama hasil.
- **Gerbang:** bila median dengan tracer melebihi 5% di atas median tanpa tracer, M1 tidak dinyatakan selesai; mitigasinya adalah penulisan buffered asinkron, pemotongan payload, dan sampling event konsol, dalam urutan tersebut.
- **Pelaporan:** hasil benchmark dicatat pada README dengan versi Lightpanda, perangkat keras, jumlah ulangan, dan angka median kedua sisi, bukan hanya persentase.

4. Spesifikasi Fitur

4.1 Fitur Inti (MVP)

F1: Startup dan koneksi tracer.

```
lp-tracer --cdp ws://127.0.0.1:9222 --output ./trace.jsonl --session "my-agent-run-001"
```

Perilaku yang diwajibkan:

- Melakukan handshake Target sesuai Bab 3.3, lalu mengirim `Page.enable`, `Network.enable`, dan `Runtime.enable` dengan `sessionId` yang benar. Domain `Log` tidak dikirim pada MVP.
- Mencatat waktu koneksi dibuat dan diputus, beserta pengenal sesi, sebagai baris JSONL bertipe `tracer.session_started` dan `tracer.session_ended`.
- Mendukung penutupan yang anggun: saat SIGINT atau SIGTERM diterima, buffer ditulis ke file (flush), baris `tracer.session_ended` ditulis, lalu proses keluar. Bila handshake Target gagal, proses keluar dengan kode bukan nol dan pesan error yang menyebut langkah handshake yang gagal.
- Memeriksa versi Lightpanda lewat endpoint HTTP `/json/version` sebelum terhubung, dan mencatat versi tersebut pada baris pertama trace. Bila versi tidak dapat dibaca, tracer tetap berjalan tetapi mencatat `lightpanda_version: "unknown"`.

F2: Cakupan pengumpulan event.

Kategori event	Sumber CDP	Field yang dicatat
Navigasi	`Page.frameNavigated`	url, frameId, timestamp tracer
Permintaan jaringan	`Network.requestWillBeSent`	requestId, url, method, resourceType, timestamp tracer
Respons jaringan	`Network.responseReceived`	status, mimeType, timestamp tracer; field `timing` CDP tidak dipakai (lihat Bab 3.2, Keputusan 4)
Selesai memuat	`Network.loadingFinished`	requestId, timestamp tracer
Kegagalan jaringan	`Network.loadingFailed`	errorText, blockedReason, timestamp tracer
Konsol	`Runtime.consoleAPICalled`	type (sebagaimana diterima), args (setelah serialisasi, redaksi, dan pemotongan), timestamp tracer
Exception JavaScript	`Runtime.exceptionThrown`	exceptionDetails, timestamp tracer; hanya bila M0b membuktikan event ini
Penanda kustom	Pemanggilan tool MCP `annotate_trace`	teks anotasi, timestamp tracer

Setiap baris JSONL memakai skema pada Lampiran A. Event yang tidak dikenal tetap ditulis dengan `type` aslinya dan `data` mentah yang sudah diredaksi, agar perilaku Lightpanda versi baru tetap terlihat tanpa mengubah tracer.

F3: Penampil trace. Aplikasi satu halaman (SPA) statis yang memuat file `.jsonl` melalui pemilih file:

- **Tampilan Timeline:** sumbu waktu horizontal, diwarnai berdasarkan jenis event. Waktu ditampilkan relatif terhadap baris pertama trace (detik ke-0).
- **Waterfall Jaringan:** waterfall permintaan yang menampilkan durasi dari stempel waktu tracer, status, dan tipe sumber daya. Kolom ukuran byte dan breakdown DNS/TLS tidak ditampilkan, karena data CDP-nya kosong pada M0.
- **Panel Konsol:** daftar log yang difilter berdasarkan level sebagaimana diterima dari CDP.
- **Pencarian:** pencarian teks penuh berdasarkan URL atau teks konsol.
- **Ekspor:** mengekspor hasil filter tampilan saat ini ke format CSV. Ekspor HAR hanya disediakan sebagai subset best-effort (tanpa breakdown timing dan tanpa ukuran), dan dilabeli demikian pada UI.

Penampil dijalankan lewat server statis lokal, misalnya `npx serve` atau `python3 -m http.server`, dari folder hasil ekspor statis. Membuka penampil langsung lewat skema `file://` tidak dijanjikan, karena pemuatan modul dan routing pada skema tersebut belum diuji.

F4: Sidecar MCP. Sidecar mengekspos tepat dua tool berikut. Tidak ada tool lain pada MVP, dan nama tool tidak boleh diubah tanpa mengubah dokumen ini.

```
{
  "name": "lp-tracer",
  "tools": [
    {
      "name": "get_trace_summary",
      "description": "Mendapatkan ringkasan sesi tracing saat ini: jumlah halaman, jumlah
permintaan, jumlah error, total durasi"
    },
    {
      "name": "annotate_trace",
      "description": "Menulis komentar kustom ke trace saat ini (untuk menandai langkah-
langkah penting)"
    }
  ]
}
```

`get_trace_summary` mengembalikan ringkasan agregat saja (jumlah dan durasi), bukan seluruh isi trace, agar konteks model tidak dipenuhi baris mentah dan rahasia yang lolos redaksi tetap terbatas. Bila file trace belum ada atau kosong, tool mengembalikan ringkasan kosong dengan penjelasan, bukan error.

4.2 Fitur Lanjutan (Iterasi Berikutnya)

- **F5:** Integrasi dengan mode `agent` Lightpanda: memetakan setiap tool call Agent ke timeline trace. Pemetaan ini memakai file `--trace` bawaan Lightpanda sebagai sumber korelasi (lihat Bab 7.3), bukan dengan menebak urutan dari timestamp semata.
- **F6:** Mode perbandingan: memuat dua file trace dan membandingkannya secara berdampingan (untuk A/B testing strategi agent yang berbeda).
- **F7:** Integrasi CI: mengeluarkan hasil “trace assertion” dalam format JUnit XML untuk pipeline pengujian otomasi.

Fitur lanjutan tidak dikerjakan sebelum MVP lulus gerbang benchmark Bab 3.7 dan pengujian M0b.

5. Bukti M0: Verifikasi Kelayakan CDP

M0 dilaksanakan pada 29 Oktober 2026 di sandbox Linux x86_64, memakai binary nightly Lightpanda `1.1.0-nightly.11275+78bbb64db` dengan telemetri dimatikan, dan skrip probe `m0-probe.mjs` tanpa dependensi (WebSocket native Node 24). Target uji adalah `https://example.com`. Skrip, hasil mentah, dan sampel event tersimpan bersama dokumen ini (`m0-probe.mjs`, `m0-result.json`, `m0-events.jsonl`, `M0-HASIL.md`). Binary nightly dihapus setelah uji; pengulangan memakai unduhan nightly yang sama atau, untuk M0b, rilis 1.0.2.

5.1 Matriks hasil

Pemeriksaan	Hasil
Handshake Target (`createTarget` + `attachToTarget`, `sessionId` `SID-1`)	Berhasil
`Page.enable`, `Network.enable`, `Runtime.enable`, `Log.enable` (dengan `sessionId`)	Diterima semua
Perintah tanpa `sessionId` (`Network.enable`, `Runtime.enable`, `Page.navigate`)	Ditolak: `BrowserContextNotLoaded`, kode -31998
`Page.frameNavigated`	1 event
`Network.requestWillBeSent`	2 event
`Network.responseReceived`	2 event
`Network.loadingFinished`	2 event
`Network.loadingFailed`	0 event (tidak ada kegagalan pada beban uji)
`Runtime.consoleAPICalled`	2 event
`Runtime.exceptionThrown`	0 event (throw lewat `Runtime.evaluate` tidak menghasilkan event)
`Log.entryAdded`	0 event (`Log.enable` diterima, tidak ada entri)
`DOM.documentUpdated`	1 event (penanda kasar)
Koneksi CDP kedua (socket open)	Berhasil dibuka tanpa mematikan koneksi pertama; attach target kedua belum diuji

5.2 Batas bukti M0

1. **Versi.** Bukti di atas berasal dari nightly 1.1.0, bukan dari 1.0.2 yang menjadi acuan dokumen. Seluruh matriks wajib diulang pada 1.0.2 (M0b) sebelum M1 dinyatakan dimulai.
2. **Exception.** Tidak terbuktinya `Runtime.exceptionThrown` pada M0 bukan bukti bahwa Lightpanda tidak pernah mengemit event tersebut. Rilis 1.0.1 mencatat penambahan emit `Runtime.exceptionThrown` untuk exception halaman yang tidak tertangkap. Uji yang benar memakai halaman yang melempar exception saat dimuat, dan itulah uji M0b.
3. **Klien kedua.** Hanya pembukaan socket kedua yang diuji. Klaim “tracer berjalan berdampingan dengan Puppeteer/Playwright” belum terbukti dan menjadi uji M0b.
4. **Timing dan ukuran.** Sampel `Network.responseReceived` menunjukkan `timing` berisi -1 untuk semua sub-field dan `encodedDataLength` 0. Batas ini dicatat sebagai fakta M0, bukan sebagai asumsi, dan menjadi dasar Keputusan 4.
5. **Level konsol.** Kedua event konsol pada M0 tercatat; pemetaan level `warn` dan `error` secara terpisah belum dikonfirmasi dan menjadi uji M0b.

Kesimpulan M0: pendekatan klien CDP eksternal **layak dilanjutkan**, dengan syarat handshake Target dan batas-batas di atas dibawa ke dalam desain, sebagaimana telah dilakukan pada Bab 3 dan 4 dokumen ini.

6. Rencana Tonggak (Milestone)

Tahap	Waktu	Deliverable
M0: Verifikasi kelayakan	Selesai 29 Oktober 2026 (nightly)	Matriks Bab 5, skrip probe, dan sampel event
M0b: Verifikasi ulang pada versi acuan	2–3 hari	Matriks Bab 5 diulang pada Lightpanda 1.0.2; uji exception pada halaman yang dimuat; uji attach target kedua dan uji berdampingan dengan satu skrip Puppeteer; uji pemetaan level konsol dan uji `Network.loadingFailed` dengan URL yang sengaja gagal. M0b adalah gerbang: bila handshake atau event inti gagal pada 1.0.2, M1 tidak dimulai dan dokumen ini direvisi.
M1: Tracer MVP	Minggu 2–3	Pengumpulan lengkap kategori event yang terverifikasi, output JSONL sesuai Lampiran A, CLI dasar sesuai F1, redaksi default sesuai Bab 3.6, dan benchmark overhead sesuai Bab 3.7
M2: Penampil MVP	Minggu 4–5	Halaman statis SvelteKit yang dapat memuat dan memvisualisasikan trace sesuai F3, termasuk ekspor CSV
M3: Sidecar MCP	Minggu 6	Server MCP mengekspos tepat dua tool (`get_trace_summary` , `annotate_trace`) sesuai F4
M4: Dokumentasi dan rilis	Minggu 7	README, contoh file trace (teredaksi), rilis npm, dan image Docker

Waktu pada tabel dihitung sejak M0b dinyatakan lulus.

7. Strategi Open Source

7.1 Kompatibilitas Lisensi

Lightpanda sendiri berlisensi **AGPL-3.0**. Tracer berjalan sebagai proses independen yang berkomunikasi dengan Lightpanda hanya melalui protokol CDP yang terbuka, tanpa menautkan kode Lightpanda. Atas dasar itu, tracer dapat memakai lisensi **Apache-2.0** atau **MIT** yang lebih ramah untuk penggunaan komersial, dan batasan ini dijelaskan secara eksplisit pada README. Keputusan lisensi final dicatat sebelum rilis pertama, dan dokumen ini bukan nasihat hukum. Bila suatu saat tracer menautkan atau menyertakan kode Lightpanda, penilaian lisensi harus diulang dari awal.

7.2 Poin Penting Panduan Kontribusi

- **Label Issue:** `good-first-issue` (komponen UI penampil), `help-wanted` (ekstensi jenis event), `documentation`.
- **Lingkungan pengembangan:** `docker-compose up` langsung menyalakan lingkungan pengembangan Lightpanda + tracer + penampil.
- **Strategi pengujian:** pengujian integrasi memakai server demo lokal sebagai target utama, agar tidak bergantung pada ketersediaan jaringan atau situs demo eksternal. Situs demo resmi Lightpanda (`demo-browser.lightpanda.io`) hanya dipakai sebagai target tambahan, bukan satu-satunya target. Setiap pengujian integrasi mengunci versi Lightpanda pada 1.0.2 dan mematikan telemetri.

7.3 Hubungan dengan Upstream

- Memantau perubahan implementasi CDP Lightpanda secara berkala, termasuk perubahan domain CDP dan catatan rilisnya.
- **Diferensiasi terhadap `agent --trace` bawaan.** Sejak rilis 1.0.1, Lightpanda menyediakan flag `agent --trace` yang menulis satu baris JSON per langkah agent (termasuk latensi pemanggilan model), dan `--trace-results` yang menambahkan teks hasil tool. Fitur bawaan tersebut mencatat langkah agent pada mode `agent` bawaan. Proyek ini berbeda pada tiga hal: (1) sumber datanya adalah event CDP (network, console, navigasi) untuk sesi CDP atau MCP apa pun, bukan hanya mode `agent` bawaan; (2) ia menyediakan penampil visual pasca-kejadian; (3) ia menyediakan ringkasan lewat sidecar MCP untuk agent eksternal. F5 pada iterasi berikutnya secara sengaja memakai file `--trace` bawaan sebagai sumber korelasi, agar kedua catatan saling melengkapi dan tidak menjadi format saingan.
- Jika Lightpanda resmi membangun fitur tracing CDP di masa depan, proyek ini dapat terus ada sebagai frontend visualisasi, atau secara aktif digabungkan ke upstream. Keputusan tersebut diambil berdasarkan catatan rilis upstream, bukan berdasarkan asumsi.

8. Risiko dan Mitigasi

Risiko	Kemungkinan	Dampak	Mitigasi
Implementasi CDP Lightpanda tidak lengkap, beberapa event hilang	Sedang	Tinggi	Verifikasi item per item pada M0 dan M0b; event yang hilang tidak diklaim sebagai fitur; kekurangan yang kritis dilengkapi lewat hook JavaScript di halaman hanya bila terbukti aman dan dicatat sebagai ketergantungan tambahan
Tracer gagal menerima event karena melewatkan handshake Target	Tinggi bila tidak diuji	Tinggi	Handshake Bab 3.3 diwajibkan dan diuji pada M0b dan M1; kegagalan handshake membuat tracer keluar dengan pesan yang jelas
Overhead performa melebihi ekspektasi	Rendah	Sedang	Penulisan event memakai buffered writer asinkron; benchmark Bab 3.7 sebagai gerbang M1
Perubahan antarmuka CDP upstream	Sedang	Sedang	Mengunci versi Lightpanda 1.0.2 untuk pengujian integrasi; memantau changelog upstream
Detail `timing` dan ukuran dari CDP kosong	Terbukti pada M0	Sedang	Waterfall dibangun dari stempel waktu tracer (Keputusan 4); tidak ada klaim breakdown DNS/TLS atau ukuran byte pada UI
Rahasia bocor ke dalam file trace atau konteks model	Sedang	Tinggi	Redaksi default Bab 3.6; body tidak direkam secara default; ringkasan MCP hanya membaca trace yang sudah disensor
Tumpang tindih dengan `agent --trace` bawaan sehingga proyek dinilai duplikatif	Sedang	Sedang	Diferensiasi Bab 7.3 dinyatakan pada README; F5 memakai file `--trace` sebagai korelasi, bukan format saingan
Kurangnya perhatian komunitas	Sedang	Rendah	Mempublikasikan di Discord dan GitHub Discussions Lightpanda, dengan menekankan penyelesaian atas titik nyeri debugging agent yang dibuktikan oleh M0
Tracer mengganggu sesi Puppeteer atau Playwright yang berjalan bersamaan	Belum teruji	Tinggi	Uji berdampingan pada M0b sebelum klaim apa pun; tracer tidak pernah menutup target milik klien lain

9. Perbedaan dengan Solusi yang Ada

Solusi	Hubungan dengan proyek ini
Chrome DevTools	Berorientasi interaksi manusia dan berat; tidak berlaku untuk Lightpanda yang tidak melakukan rendering
Playwright Trace Viewer	Terikat ekosistem Playwright; proyek ini langsung terhubung ke CDP dan mencakup skenario MCP dan agent eksternal
Log native Lightpanda	Hanya log teks, tanpa aliran event terstruktur dan tanpa visualisasi
`agent --trace` bawaan Lightpanda (sejak 1.0.1)	Mencatat langkah agent pada mode `agent` bawaan; proyek ini mencatat event CDP untuk sesi apa pun, ditambah penampil dan ringkasan MCP. Keduanya saling melengkapi (lihat Bab 7.3)

10. Langkah Selanjutnya

Langkah selanjutnya adalah **M0b** (Bab 6): mengulang matriks M0 pada Lightpanda 1.0.2, menguji exception pada halaman yang sedang dimuat, menguji attach target kedua dan berdampingan dengan Puppeteer, serta menguji kegagalan jaringan dan pemetaan level konsol. Bila M0b lulus, repositori resmi dibuat dan M1 dimulai. Bila M0b gagal pada event inti, dokumen ini direvisi sebelum pekerjaan M1 apa pun dimulai.

Lampiran A: Skema JSONL dan Contoh Baris

A.1 Skema

Setiap baris adalah satu objek JSON dengan field berikut:

Field	Tipe	Keterangan
`ts`	string (ISO 8601, UTC)	Waktu diterimanya event menurut jam tracer
`mono_ms`	number	Waktu monotonik tracer dalam milidetik sejak sesi dimulai; dipakai untuk menghitung durasi dan urutan
`seq`	integer	Nomor urut baris dalam satu file, mulai dari 1
`session`	string	Pengenal sesi dari flag `--session`; bila tidak diberikan, dibuat otomatis dan dicatat pada baris pertama
`lightpanda_version`	string	Versi Lightpanda dari `/json/version`, atau `"unknown"`
`type`	string	Jenis event, memakai nama method CDP apa adanya (misalnya `Network.requestWillBeSent`), atau nama bertipe `tracer.*` untuk baris internal tracer
`data`	object	Payload event setelah redaksi dan pemotongan sesuai Bab 3.6

Baris internal tracer memakai `type` `tracer.session_started`, `tracer.session_ended`, dan `tracer.annotation` (untuk anotasi dari `annotate_trace`). Field `data` pada baris internal tidak pernah berisi payload CDP mentah.

A.2 Contoh baris (sintetis, sudah disensor)

```
{"ts":"2026-10-29T05:44:05.153Z","mono_ms":0,"seq":1,"session":"my-agent-run-
001","lightpanda_version":"1.0.2","type":"tracer.session_started","data":
{"cdp":"ws://127.0.0.1:9222"}}
{"ts":"2026-10-29T05:44:06.102Z","mono_ms":949,"seq":2,"session":"my-agent-run-
001","lightpanda_version":"1.0.2","type":"Network.requestWillBeSent","data":
{"requestId":"LID-
0000000001","url":"https://example.com/","method":"GET","type":"Document"}}
{"ts":"2026-10-29T05:44:06.610Z","mono_ms":1457,"seq":3,"session":"my-agent-run-
001","lightpanda_version":"1.0.2","type":"Network.responseReceived","data":
{"requestId":"LID-
0000000001","url":"https://example.com/","status":200,"mimeType":"text/html"}}
{"ts":"2026-10-29T05:44:07.004Z","mono_ms":1851,"seq":4,"session":"my-agent-run-
001","lightpanda_version":"1.0.2","type":"Runtime.consoleAPICalled","data":
{"type":"log","args":["LP-TRACER-PROBE-LOG"]}}
```

Contoh di atas memakai stempel waktu dan pengenal yang disederhanakan untuk keterbacaan. Implementasi wajib memakai nilai nyata dari sesi yang direkam.

Lampiran B: Sumber

1. Lightpanda, “Lightpanda 1.0: Out of Beta and Ready for Production”, blog Lightpanda, 2 Oktober 2026 — 1,7 juta sub-tes Web Platform Tests yang lulus dan CORS default. https://lightpanda.io/blog/
2. Lightpanda, “From Local to Real World Benchmarks”, blog Lightpanda, 27 Januari 2026 — benchmark 933 halaman, 25 tugas paralel, 123 MB berbanding 2 GB, sekitar 5 detik berbanding 46 detik. https://lightpanda.io/blog/posts/from-local-to-real-world-benchmarks
3. Lightpanda, Release 1.0.1, GitHub `lightpanda-io/browser` — penambahan emit `Runtime.exceptionThrown`, flag `agent --trace` dan `--trace-results`, serta telemetri pemanggilan tool. https://github.com/lightpanda-io/browser/releases/tag/1.0.1
4. Lightpanda, Release 1.0.2, GitHub `lightpanda-io/browser`, 23 Oktober 2026 — versi acuan dokumen ini. https://github.com/lightpanda-io/browser/releases/tag/1.0.2
5. Bukti M0 proyek ini, 29 Oktober 2026 — `m0-probe.mjs`, `m0-result.json`, `m0-events.jsonl`, dan `M0-HASIL.md`, tersimpan bersama dokumen ini. Bukti ini memakai nightly 1.1.0 dan wajib diulang pada 1.0.2 (M0b).

Dokumen ini disusun sebagai bagian dari perancangan proyek open source turunan Lightpanda. Untuk pertanyaan dan kontribusi, silakan buka Issue di repositori proyek setelah repositori dibuat pada akhir M0b.